```
!pip install scikit-learn pandas numpy matplotlib seaborn -q

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, classification_rep
from sklearn.neural_network import MLPClassifier
import warnings
warnings.filterwarnings('ignore')

print("✓ All libraries imported successfully!")

# ============================================================
# PART 1: MOONS DATASET - PERCEPTRON VS MULTI-LAYER NEURAL NET
# ============================================================

print("\n" + "=" * 70)
print("PART 1: MOONS DATASET ANALYSIS")
print("=" * 70)

# Generate moons dataset
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"\nDataset Shape: {X.shape}")
print(f"Training samples: {X_train.shape[0]}")
print(f"Testing samples: {X_test.shape[0]}")

# Visualize the moons dataset
plt.figure(figsize=(10, 6))
plt.scatter(X[y==0][:, 0], X[y==0][:, 1], c='blue', label='Cla
plt.scatter(X[y==1][:, 0], X[y==1][:, 1], c='red', label='Clas
plt.title('Moons Dataset (Non-linearly Separable)', fontsize=1
plt.xlabel('Feature 1')
```

```
plt.ylabel('Feature 2')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# ------------------------------------------------------------
# Model 1: Perceptron (Linear Classifier)
# ------------------------------------------------------------
print("\n" + "-" * 70)
print("Training Perceptron...")
print("-" * 70)

perceptron = Perceptron(max_iter=1000, random_state=42)
perceptron.fit(X_train_scaled, y_train)
y_pred_perceptron = perceptron.predict(X_test_scaled)

perceptron_accuracy = accuracy_score(y_test, y_pred_perceptron
print(f"Perceptron Accuracy: {perceptron_accuracy:.4f}")

# ------------------------------------------------------------
# Model 2: Multi-layer Feedforward Neural Network
# ------------------------------------------------------------
print("\n" + "-" * 70)
print("Training Multi-layer Neural Network...")
print("-" * 70)

mlp = MLPClassifier(hidden_layer_sizes=(10, 10), activation='r
                    max_iter=1000, random_state=42, learning_r
mlp.fit(X_train_scaled, y_train)
y_pred_mlp = mlp.predict(X_test_scaled)

mlp_accuracy = accuracy_score(y_test, y_pred_mlp)
print(f"MLP Accuracy: {mlp_accuracy:.4f}")

# ------------------------------------------------------------
# Comparison Table for Part 1
# ------------------------------------------------------------
print("\n" + "=" * 70)
print("COMPARISON TABLE: PERCEPTRON VS MULTI-LAYER NEURAL NETW
print("=" * 70)

comparison_data = {
    'Metric': [
        'Accuracy',
        'Model Complexity',
        'Training Time',
```

```
            'Training Time',
            'Ability to Handle Non-linearity',
            'Number of Parameters'
        ],
        'Perceptron': [
            f'{perceptron_accuracy:.4f}',
            'Low (Linear Decision Boundary)',
            'Fast',
            'Cannot handle non-linear separation',
            f'{perceptron.coef_.size + 1}'
        ],
        'Multi-layer Neural Network': [
            f'{mlp_accuracy:.4f}',
            'High (Non-linear Decision Boundary)',
            'Moderate',
            'Excellent non-linear separation',
            f'{sum([w.size for w in mlp.coefs_]) + sum([b.size for
        ]
    }

    comparison_df = pd.DataFrame(comparison_data)
    print("\n")
    print(comparison_df.to_string(index=False))

    # Observations
    print("\n" + "=" * 70)
    print("OBSERVATIONS:")
    print("=" * 70)
    print(f"1. Accuracy: MLP outperforms Perceptron by {(mlp_accur
          f"due to its ability to learn non-linear decision bounda
    print(f"\n2. Model Complexity: Perceptron is simpler but fails
          f"while MLP handles complex patterns effectively.")
    print(f"\n3. Non-linearity: The moons dataset is non-linearly
          f"inadequate while the MLP excels.")

    # Decision boundary visualization
    def plot_decision_boundary(model, X, y, title, scaler):
        h = 0.02
        x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
        y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
        xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange

        Z = model.predict(scaler.transform(np.c_[xx.ravel(), yy.ra
        Z = Z.reshape(xx.shape)

        plt.contourf(xx, yy, Z, alpha=0.3, cmap='RdYlBu')
        plt.scatter(X[y==0][:, 0], X[y==0][:, 1], c='blue', label=
```

```
        plt.scatter(X[y==1][:, 0], X[y==1][:, 1], c='red', label='
        plt.title(title, fontsize=12, fontweight='bold')
        plt.xlabel('Feature 1')
        plt.ylabel('Feature 2')
        plt.legend()

    plt.figure(figsize=(15, 5))
    plt.subplot(1, 2, 1)
    plot_decision_boundary(perceptron, X_test, y_test, 'Perceptron
    plt.subplot(1, 2, 2)
    plot_decision_boundary(mlp, X_test, y_test, 'Multi-layer NN De
    plt.tight_layout()
    plt.show()

    # ============================================================
    # PART 2: MANUAL BACKPROPAGATION WITH UPLOADED DATASETS
    # ============================================================

    print("\n\n" + "=" * 70)
    print("PART 2: MANUAL BACKPROPAGATION ON KAGGLE DATASETS")
    print("=" * 70)

    # ============================================================
    # MANUAL BACKPROPAGATION IMPLEMENTATION
    # ============================================================

    class ManualNeuralNetwork:
        """Multi-layer Neural Network with Manual Backpropagation"

        def __init__(self, input_size, hidden_size, output_size, a
                     learning_rate=0.01, epochs=1000):
            self.input_size = input_size
            self.hidden_size = hidden_size
            self.output_size = output_size
            self.activation_type = activation
            self.learning_rate = learning_rate
            self.epochs = epochs

            # Initialize weights with Xavier initialization
            self.W1 = np.random.randn(input_size, hidden_size) * n
            self.b1 = np.zeros((1, hidden_size))
            self.W2 = np.random.randn(hidden_size, output_size) *
            self.b2 = np.zeros((1, output_size))

            self.losses = []

        def activation(self, z):
```

```
    def activation(self, z):
        """Activation function"""
        if self.activation_type == 'sigmoid':
            return 1 / (1 + np.exp(-np.clip(z, -500, 500)))
        elif self.activation_type == 'tanh':
            return np.tanh(z)
        elif self.activation_type == 'relu':
            return np.maximum(0, z)

    def activation_derivative(self, z):
        """Derivative of activation function"""
        if self.activation_type == 'sigmoid':
            sig = self.activation(z)
            return sig * (1 - sig)
        elif self.activation_type == 'tanh':
            return 1 - np.tanh(z) ** 2
        elif self.activation_type == 'relu':
            return (z > 0).astype(float)

    def softmax(self, z):
        """Softmax activation for output layer"""
        exp_z = np.exp(z - np.max(z, axis=1, keepdims=True))
        return exp_z / np.sum(exp_z, axis=1, keepdims=True)

    def forward(self, X):
        """Forward propagation"""
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = self.activation(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = self.softmax(self.z2)
        return self.a2

    def backward(self, X, y, output):
        """Backpropagation"""
        m = X.shape[0]

        # Output layer gradients
        dz2 = output - y
        dW2 = np.dot(self.a1.T, dz2) / m
        db2 = np.sum(dz2, axis=0, keepdims=True) / m

        # Hidden layer gradients
        da1 = np.dot(dz2, self.W2.T)
        dz1 = da1 * self.activation_derivative(self.z1)
        dW1 = np.dot(X.T, dz1) / m
        db1 = np.sum(dz1, axis=0, keepdims=True) / m
```

```
            # Update weights
            self.W2 -= self.learning_rate * dW2
            self.b2 -= self.learning_rate * db2
            self.W1 -= self.learning_rate * dW1
            self.b1 -= self.learning_rate * db1

        def compute_loss(self, y_true, y_pred):
            """Cross-entropy loss"""
            m = y_true.shape[0]
            log_likelihood = -np.log(y_pred[range(m), y_true.argma
            return np.sum(log_likelihood) / m

        def fit(self, X, y, verbose=False):
            """Train the network"""
            for epoch in range(self.epochs):
                # Forward propagation
                output = self.forward(X)

                # Compute loss
                loss = self.compute_loss(y, output)
                self.losses.append(loss)

                # Backpropagation
                self.backward(X, y, output)

                if verbose and (epoch + 1) % 100 == 0:
                    print(f'Epoch {epoch + 1}/{self.epochs}, Loss:

        def predict(self, X):
            """Make predictions"""
            output = self.forward(X)
            return np.argmax(output, axis=1)

        def score(self, X, y):
            """Calculate accuracy"""
            predictions = self.predict(X)
            y_true = np.argmax(y, axis=1)
            return np.mean(predictions == y_true)

    # ============================================================
    # DATASET 1: IRIS DATASET
    # ============================================================

    print("\n" + "=" * 70)
    print("DATASET 1: IRIS")
    print("=" * 70)
```

```
# Load Iris dataset from the specified path
iris_df = pd.read_csv('/content/Iris.csv')
print(f"\nIris Dataset Shape: {iris_df.shape}")
print(f"Columns: {list(iris_df.columns)}")

# Handle different possible column names
if 'Species' in iris_df.columns:
    species_col = 'Species'
elif 'species' in iris_df.columns:
    species_col = 'species'
else:
    species_col = iris_df.columns[-1]

print(f"Classes: {iris_df[species_col].unique()}")

# Prepare data - take all columns except first (Id) and last (
if 'Id' in iris_df.columns or 'id' in iris_df.columns:
    X_iris = iris_df.iloc[:, 1:-1].values
else:
    X_iris = iris_df.iloc[:, :-1].values

y_iris = iris_df[species_col].values

# Encode labels
le_iris = LabelEncoder()
y_iris_encoded = le_iris.fit_transform(y_iris)
y_iris_onehot = np.eye(len(np.unique(y_iris_encoded)))[y_iris_

# Split and scale
X_iris_train, X_iris_test, y_iris_train, y_iris_test = train_t
    X_iris, y_iris_onehot, test_size=0.3, random_state=42)

scaler_iris = StandardScaler()
X_iris_train_scaled = scaler_iris.fit_transform(X_iris_train)
X_iris_test_scaled = scaler_iris.transform(X_iris_test)

print(f"Training samples: {X_iris_train.shape[0]}")
print(f"Testing samples: {X_iris_test.shape[0]}")

# ============================================================
# DATASET 2: WINE QUALITY DATASET
# ============================================================

print("\n" + "=" * 70)
print("DATASET 2: WINE QUALITY")
print("=" * 70)
```

```
print("=" * 70)

# Load Wine dataset from the specified path
wine_df = pd.read_csv('/content/winequality-red.csv', sep=';')
print(f"\nWine Dataset Shape: {wine_df.shape}")

# Prepare data (classify wine quality into 3 categories: Low,
X_wine = wine_df.iloc[:, :-1].values
y_wine = wine_df['quality'].values

# Convert to 3 classes
y_wine_binned = np.zeros_like(y_wine)
y_wine_binned[y_wine <= 5] = 0  # Low quality
y_wine_binned[(y_wine > 5) & (y_wine <= 6)] = 1  # Medium qual
y_wine_binned[y_wine > 6] = 2  # High quality

y_wine_onehot = np.eye(3)[y_wine_binned]

print(f"Quality Classes Distribution:")
print(f"  Low (≤5): {np.sum(y_wine_binned == 0)}")
print(f"  Medium (6): {np.sum(y_wine_binned == 1)}")
print(f"  High (>6): {np.sum(y_wine_binned == 2)}")

# Split and scale
X_wine_train, X_wine_test, y_wine_train, y_wine_test = train_t
    X_wine, y_wine_onehot, test_size=0.3, random_state=42)

scaler_wine = StandardScaler()
X_wine_train_scaled = scaler_wine.fit_transform(X_wine_train)
X_wine_test_scaled = scaler_wine.transform(X_wine_test)

print(f"Training samples: {X_wine_train.shape[0]}")
print(f"Testing samples: {X_wine_test.shape[0]}")

# ============================================================
# EXPERIMENT WITH DIFFERENT SETTINGS
# ============================================================

print("\n\n" + "=" * 70)
print("RUNNING EXPERIMENTS WITH DIFFERENT SETTINGS")
print("=" * 70)

# Settings to compare
settings = [
    {'name': 'Activation Function: Sigmoid', 'activation': 'si
    {'name': 'Activation Function: Tanh', 'activation': 'tanh'
    {'name': 'Activation Function: ReLU', 'activation': 'relu'
```

```
    {'name': 'Hidden Layer Size: 5 neurons', 'activation': 're
    {'name': 'Hidden Layer Size: 20 neurons', 'activation': 'r
    {'name': 'Learning Rate: 0.001', 'activation': 'relu', 'hi
    {'name': 'Learning Rate: 0.1', 'activation': 'relu', 'hidd
]

results = []

for i, setting in enumerate(settings):
    print(f"\n{'-' * 70}")
    print(f"Experiment {i+1}/7: {setting['name']}")
    print(f"{'-' * 70}")

    # Train on Iris
    print("Training on Iris dataset...")
    nn_iris = ManualNeuralNetwork(
        input_size=X_iris_train_scaled.shape[1],
        hidden_size=setting['hidden_size'],
        output_size=3,
        activation=setting['activation'],
        learning_rate=setting['learning_rate'],
        epochs=setting['epochs']
    )
    nn_iris.fit(X_iris_train_scaled, y_iris_train)
    iris_accuracy = nn_iris.score(X_iris_test_scaled, y_iris_t

    # Train on Wine
    print("Training on Wine dataset...")
    nn_wine = ManualNeuralNetwork(
        input_size=X_wine_train_scaled.shape[1],
        hidden_size=setting['hidden_size'],
        output_size=3,
        activation=setting['activation'],
        learning_rate=setting['learning_rate'],
        epochs=setting['epochs']
    )
    nn_wine.fit(X_wine_train_scaled, y_wine_train)
    wine_accuracy = nn_wine.score(X_wine_test_scaled, y_wine_t

    results.append({
        'Setting': setting['name'],
        'Iris Accuracy': f'{iris_accuracy:.4f}',
        'Wine Accuracy': f'{wine_accuracy:.4f}'
    })

    print(f"✓ Iris Accuracy: {iris_accuracy:.4f}")
```

```
    print(f"✓ Wine Accuracy: {wine_accuracy:.4f}")

# ============================================================
# COMPARISON TABLE FOR PART 2
# ============================================================

print("\n\n" + "=" * 70)
print("SETTINGS COMPARISON TABLE")
print("=" * 70)

results_df = pd.DataFrame(results)
print("\n")
print(results_df.to_string(index=False))

# ============================================================
# VISUALIZATIONS
# ============================================================

print("\n\n" + "=" * 70)
print("GENERATING VISUALIZATIONS")
print("=" * 70)

# Plot accuracy comparison
plt.figure(figsize=(14, 6))

# Extract activation function results
activation_results = results[:3]
activation_names = [r['Setting'].split(': ')[1] for r in activ
iris_acc_act = [float(r['Iris Accuracy']) for r in activation_
wine_acc_act = [float(r['Wine Accuracy']) for r in activation_

x = np.arange(len(activation_names))
width = 0.35

plt.subplot(1, 2, 1)
plt.bar(x - width/2, iris_acc_act, width, label='Iris', alpha=
plt.bar(x + width/2, wine_acc_act, width, label='Wine', alpha=
plt.xlabel('Activation Function', fontweight='bold')
plt.ylabel('Accuracy', fontweight='bold')
plt.title('Accuracy Comparison: Activation Functions', fontwei
plt.xticks(x, activation_names)
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.ylim([0, 1.1])

# Extract learning rate results
lr_results = [results[2]] + results[5:7]  # Include ReLU base
```

```
lr_results = [results[2]] + results[5:7]  # Include ReLU base
lr_names = [r['Setting'].split(': ')[1] if 'Learning Rate' in
iris_acc_lr = [float(r['Iris Accuracy']) for r in lr_results]
wine_acc_lr = [float(r['Wine Accuracy']) for r in lr_results]

x2 = np.arange(len(lr_names))

plt.subplot(1, 2, 2)
plt.bar(x2 - width/2, iris_acc_lr, width, label='Iris', alpha=
plt.bar(x2 + width/2, wine_acc_lr, width, label='Wine', alpha=
plt.xlabel('Learning Rate', fontweight='bold')
plt.ylabel('Accuracy', fontweight='bold')
plt.title('Accuracy Comparison: Learning Rates', fontweight='b
plt.xticks(x2, lr_names, rotation=15)
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.ylim([0, 1.1])

plt.tight_layout()
plt.show()

# Hidden layer size comparison
plt.figure(figsize=(10, 6))
hidden_results = [results[3], results[2], results[4]]  # 5, 10
hidden_names = ['5 neurons', '10 neurons (baseline)', '20 neur
iris_acc_hidden = [float(r['Iris Accuracy']) for r in hidden_r
wine_acc_hidden = [float(r['Wine Accuracy']) for r in hidden_r

x3 = np.arange(len(hidden_names))
plt.bar(x3 - width/2, iris_acc_hidden, width, label='Iris', al
plt.bar(x3 + width/2, wine_acc_hidden, width, label='Wine', al
plt.xlabel('Hidden Layer Size', fontweight='bold')
plt.ylabel('Accuracy', fontweight='bold')
plt.title('Accuracy Comparison: Hidden Layer Sizes', fontweigh
plt.xticks(x3, hidden_names)
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.ylim([0, 1.1])
plt.tight_layout()
plt.show()

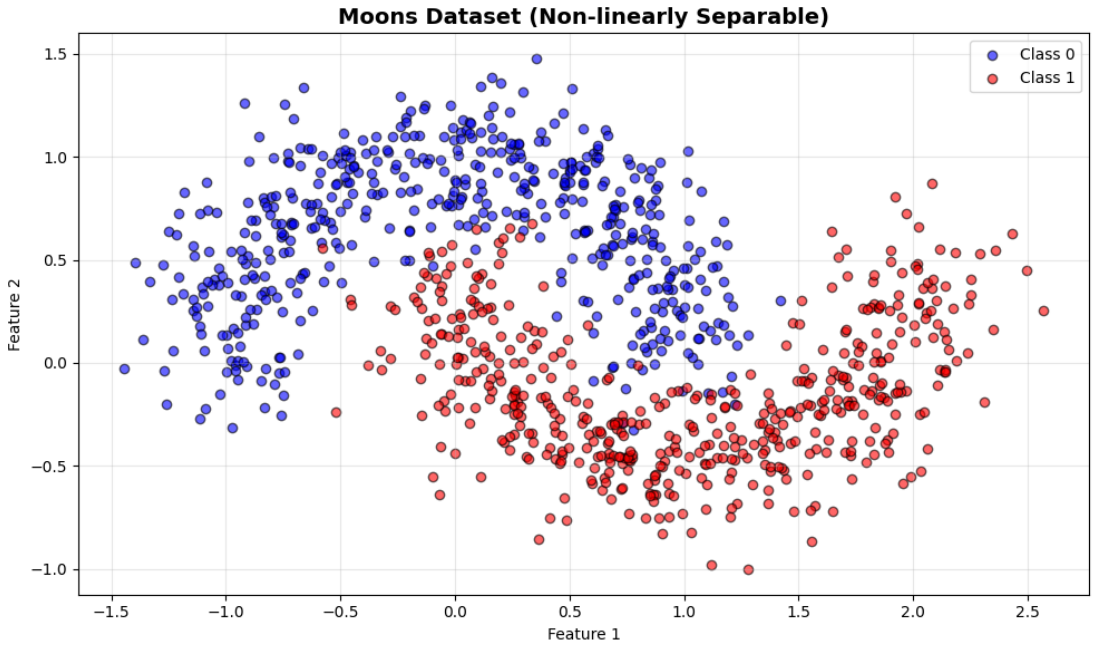
print("\n✓ All experiments completed successfully!")
print("\n" + "=" * 70)
print("SUMMARY")
print("=" * 70)
print("✓ Part 1: Successfully compared Perceptron vs Multi-lay
print("✓ Part 2: Successfully implemented manual backpropagati
```

```
print("           on Iris and Wine Quality datasets")
print("\nAll comparison tables and observations have been gene
print("=" * 70)
```

```
✓ All libraries imported successfully!

======================================================================
PART 1: MOONS DATASET ANALYSIS
======================================================================

Dataset Shape: (1000, 2)
Training samples: 700
Testing samples: 300
```




```
----------------------------------------------------------------------
Training Perceptron...
----------------------------------------------------------------------
Perceptron Accuracy: 0.8467

----------------------------------------------------------------------
Training Multi-layer Neural Network...
----------------------------------------------------------------------
MLP Accuracy: 0.9767

======================================================================
COMPARISON TABLE: PERCEPTRON VS MULTI-LAYER NEURAL NETWORK
======================================================================

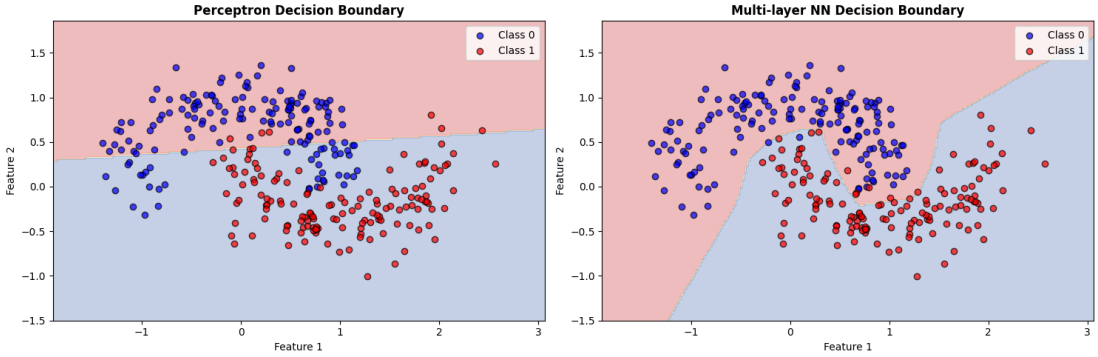
                                Metric                                Perce
                              Accuracy                                    (
                      Model Complexity      Low (Linear Decision Boun
```

```
                Model Complexity      Low (Linear Decision Boun
                   Training Time
Ability to Handle Non-linearity Cannot handle non-linear separ
           Number of Parameters

================================================================
OBSERVATIONS:
================================================================
1. Accuracy: MLP outperforms Perceptron by 13.00% due to its a

2. Model Complexity: Perceptron is simpler but fails on non-li

3. Non-linearity: The moons dataset is non-linearly separable,
```




```
================================================================
PART 2: MANUAL BACKPROPAGATION ON KAGGLE DATASETS
================================================================

================================================================
DATASET 1: IRIS
================================================================

Iris Dataset Shape: (150, 6)
Columns: ['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthC
```